

可测试性报告 (Lab1.1 → Lab1.2)

本部分按讨论课要求聚焦“可测试性”，内容简明，每部分提炼 2 点亮点与 2 点改进建议，并结合当前代码迭代。

1. 架构设计对单元测试成本的降低

- **依赖注入 + 接口抽象**：workspaceService 依赖 FileSystem、LogService、SpellCheckService、StatisticsService 等接口/服务，测试可替换实现，减少环境成本。
- **Lab1.1 → Lab1.2 的变化**：构造函数新增 SpellCheckService 与 StatisticsService 注入，说明核心能力从“内嵌实现”转为“可替换实现”。这直接降低了测试引入真实依赖的成本。
- **降低测试准备成本**：核心服务通过构造函数注入，测试可只注入必要依赖并用轻量替身替代，避免引入真实文件系统或网络调用。
- 代码示例（迭代前 / 迭代后）：

```
1 // 迭代前：依赖较少，拼写检查/统计未注入
2 public workspaceService(FileSystem fileSystem,
3                           WorkspacePersistence persistence,
4                           EventBus eventBus,
5                           LogService logService,
6                           Console console) {
7     this.fileSystem = fileSystem;
8     this.persistence = persistence;
9     this.logService = logService;
10    this.console = console;
11 }
12
13 // 迭代后：注入 SpellCheckService 与 StatisticsService
14 public workspaceService(FileSystem fileSystem,
15                           WorkspacePersistence persistence,
16                           EventBus eventBus,
17                           LogService logService,
18                           Console console,
19                           SpellCheckService spellCheckService,
20                           StatisticsService statistics) {
21     this.fileSystem = fileSystem;
22     this.persistence = persistence;
23     this.logService = logService;
24     this.console = console;
25     this.eventBus = eventBus;
26     this.spellCheckService = spellCheckService;
27     this.statistics = statistics;
28 }
```

- **适配器隔离第三方**：拼写检查通过 SpellCheckService 抽象与 SpellCheckServiceFactory 注入，切换实现无需改动编辑器逻辑。
- **Lab1.1 → Lab1.2 的变化**：拼写检查从“固定词典实现”升级为“按配置选择实现”，因此更容易用 Mock 替换与验证异常分支。
- **可替换实现便于 Mock**：接口抽象让测试用伪实现覆盖“拼写检查返回值”，无需真实请求或第三方库，保证单测稳定。

- 代码示例（迭代前 / 迭代后）：

```
1 // 迭代前：在装配层直接绑定具体实现
2 SpellCheckService spellCheckService =
  DictionarySpellCheckAdapter.defaultEnglish();
3
4 // 迭代后：通过工厂按配置切换实现
5 public static SpellCheckService fromSystemProperties() {
6     String provider = System.getProperty(PROVIDER_KEY,
7     "dictionary").trim().toLowerCase();
8     if ("languageTool-http".equals(provider)) {
9         String endpoint = System.getProperty(ENDPOINT_KEY,
10         "https://api.languagetool.org/v2/check");
11         String language = System.getProperty(LANGUAGE_KEY, "en-US");
12         return new LanguageToolHttpAdapter(URI.create(endpoint),
13         HttpClient.newHttpClient(), language);
14     }
15     return DictionarySpellCheckAdapter.defaultEnglish();
16 }
```

2. 测试隔离手段与差异

- **可控 I/O 与时间**：FakeConsole、FakeClock 让交互与时钟可预测，减少不稳定因素。
- **Lab1.1 → Lab1.2 的变化**：新增统计功能后，时间控制变得更关键，FakeClock 使“编辑时长累计”变成可断言行为。
- **减少非业务干扰**：输入输出与时间统一替身处理，测试只关注业务行为，不受外部环境影响。
- 代码示例（迭代前 / 迭代后）：

```
1 // 迭代前：测试中难以控制输入与时间
2 Console console = new SystemConsole();
3 Clock clock = new SystemClock();
4
5 // 迭代后：可控输入与固定时间
6 @Override
7 public String readLine() {
8     return inputs.isEmpty() ? "n" : inputs.removeFirst();
9 }
10
11 @Override
12 public LocalDateTime now() {
13     return now;
14 }
```

- **分层回归覆盖**：CommandChecklistRegressionTest 覆盖新增 XML 命令链路，保证跨模块回归。
- **Lab1.1 → Lab1.2 的变化**：回归测试从“仅文本命令”扩展到 XML 命令，验证命令注册与解析的完整性。
- **回归与扩展兼容**：命令清单测试能快速发现“新增命令未注册/参数解析缺失”的问题，降低新增模块的回归风险。
- 代码示例（迭代前 / 迭代后）：

```

1 // 迭代前：仅验证文本命令链路
2 assertOk(reg.execute("append \"Hello\""));
3 assertOk(reg.execute("insert 1:1 \"X\""));
4
5 // 迭代后：加入 XML 命令链路
6 assertOk(reg.execute("append-child book book1 root \"Hello\""));
7 assertOk(reg.execute("insert-before book book0 book1 \"\""));
8 assertOk(reg.execute("edit-text book1 \"Title\""));
9 assertOk(reg.execute("edit-id book1 book001"));
10 assertOk(reg.execute("xml-tree"));
11 assertOk(reg.execute("delete book001"));

```

3. 测试文件分析（层次与覆盖）

- **单元层：** `TextEditorTest` 覆盖编辑与撤销重做边界，保障文本编辑核心行为。
- **Lab1.1 → Lab1.2 的变化：** 新增拼写检查装饰器后，单元测试增加了“拼写输出格式”断言，确保输出符合文档要求。
- **边界行为可控：** 多行插入、越界删除等关键边界都有断言，减少“静默失败”。
- 代码示例（迭代前 / 迭代后）：

```

1 // 迭代前：仅测试单行插入
2 assertEquals("ok", ed.insert(new LineCol(1, 2), "X"));
3 assertEquals(List.of("aXbc"), ed.lines());
4
5 // 迭代后：补充多行插入边界
6 assertEquals("ok", ed.insert(new LineCol(1, 3), "X\nY"));
7 assertEquals(List.of("abX", "Ycd"), ed.lines());

```

- **集成层：** `WorkspaceServiceTest` 验证 load/save/restore 等关键流程。
- **Lab1.1 → Lab1.2 的变化：** 工作区构造与恢复过程需包含统计与拼写服务，集成测试覆盖链路更完整。
- **关键链路覆盖：** 覆盖“退出 -> 快照持久化 -> 恢复”的主流程，保证核心功能可用。
- 代码示例（迭代前 / 迭代后）：

```

1 // 迭代前：仅验证 load/save
2 assertEquals("ok", reg1.execute("load \"\" + file + "\"").output());
3 assertEquals("ok", reg1.execute("save").output());
4
5 // 迭代后：补充 exit/restore 流程
6 assertEquals("ok", reg1.execute("load \"\" + file + "\"").output());
7 assertEquals("ok", reg1.execute("append \"New line\"").output());
8 ExecutionResult exit = reg1.execute("exit");
9 assertTrue(exit.shouldExit());

```

4. 改进点

- **XML 专属异常覆盖不足：** 如 ID 冲突、根节点删除、空文档 `xml-tree`，需要补充单元测试。
- **Lab1.1 → Lab1.2 的变化：** XML 相关逻辑新增后，异常分支数量上升，但测试仍偏主流程，需补齐异常路径。

- **异常分支用例不足**：缺少“非法参数/空输入”的命令级测试，容易遗漏报错格式一致性。
- **统计/拼写切换缺独立用例**：`StatisticsService` 和拼写切换工厂缺少独立测试，影响回归稳定性。
- **插件边界验证不足**：XML 与文本命令混用时的“不支持”提示缺少集中测试。

5. 对 Lab1.2 扩展性的结论

- 通过插件包与工厂注入，新增编辑器/拼写实现成本低，符合开闭原则。
- **Lab1.1 → Lab1.2 的变化**：新增 XML 插件与统计模块后，核心工作区未被重写，只需扩展装配与路由，说明整体架构可扩展。
- 统计与拼写检查均为横切功能，通过事件与适配器解耦，新增功能不需要侵入核心编辑器。
- 当前迭代已完成的代码改动（用代码说明）：

```
1 // XML 插件核心入口（编辑器与树结构）
2 public final class XmlEditor implements Editor { /* ... */ }
3 public final class XmlElement implements XmlNode { /* ... */ }
4
5 // 统计模块：通过事件解耦
6 public final class StatisticsService {
7     public StatisticsService(EventBus bus, Clock clock) {
8         bus.subscribe(EditorActivatedEvent.class, ev ->
9             activate(ev.file()));
10        bus.subscribe(EditorDeactivatedEvent.class, ev ->
11            deactivate(ev.file()));
12        bus.subscribe(EditorClosedEvent.class, ev -> reset(ev.file()));
13    }
14 }
15
16 // 拼写检查可切换：工厂注入
17 spellCheckService spellCheckService =
18     SpellCheckServiceFactory.fromSystemProperties();
```

6. 大模型使用感想

- **效率提升**：在命令设计、边界梳理与文档整理上能快速给出结构化草稿，减少低价值重复劳动。
- **质量可控**：对关键接口、异常处理与输出格式仍需人工校验，避免“看似合理但与要求不一致”的问题。
- **协作方式**：最有效的方式是提供清晰约束与示例，再让模型补全与对齐，形成“人定标准、模型补细节”的流程。